Customer Support Agent

27 October 2026 | Advanced Cloud Architecture & DevOps CI/CD Pipeline Process

Python + React + Docker + Terraform + Azure + CI/CD

- Receive a customer query and draft a grounded answer.
- Assign it to a human specialist for review.
- Deliver only the approved response back to the customer.

Imoh Etuk | MMAUG | Learn. Innovate. Build.

Two sessions, one learning journey

- 17 October foundations: Python behavior, Docker runtime, Terraform resource intent.
- 27 October: rebuild those foundations into a two-web-app support agent.
- Add React, Blob-backed data, App Service, managed identity and optional Foundry.
- Trace one immutable image through tests, approvals, release and verification.
- This edition replaces the previous app examples; current application hosting is App Service only.

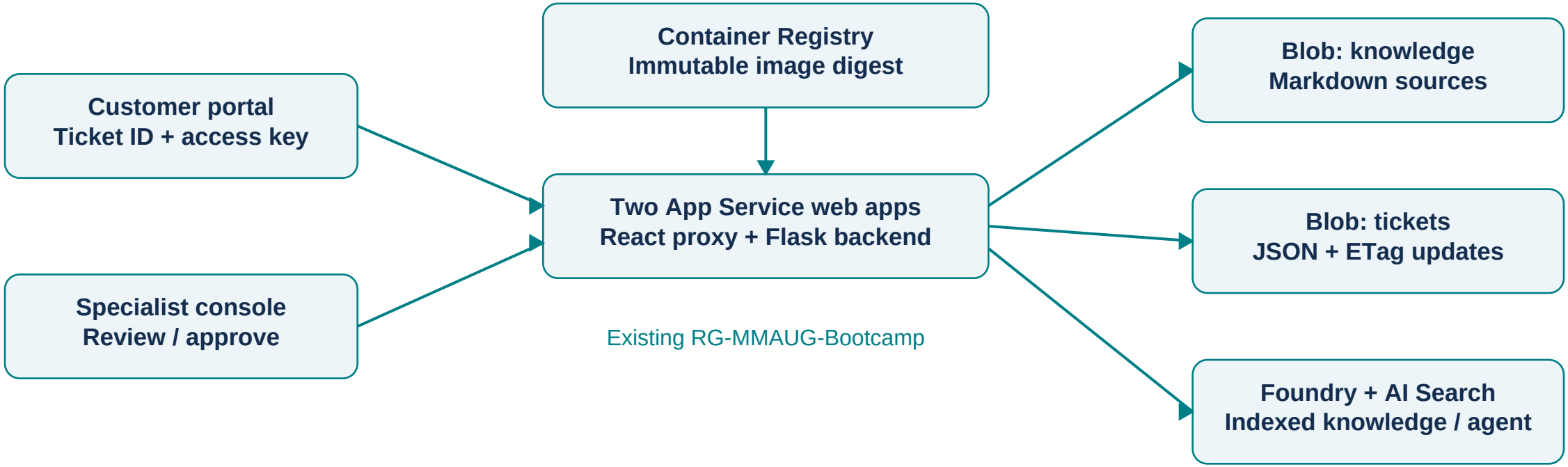
Local labs need no Azure subscription. Live Azure exercises incur costs.

The human-approval contract

- Customer submits a question in the React portal.
- Backend classifies it, searches knowledge, drafts a reply and assigns a specialist.
- Specialist inspects the draft, edits and approves it, or reassigns the ticket.
- Until approval, the customer sees status only, never the draft.
- The approved reply appears in the portal. Email delivery is not implemented.

AI drafts; people decide. A green deployment does not prove an answer is correct.

Azure architecture



Two web apps in existing RG-MMAUG-Bootcamp; Blob data and indexed retrieval have separate roles.

A map of the repository

```
support-agent/backend/   Flask API, agent, storage and tests
support-agent/frontend/  React customer + specialist views
support-agent/knowledge/ Markdown knowledge source
support-agent/terraform/ Local Docker-provider lab
infra/azure/             Two apps, ACR, Foundry and Search
.github/workflows/       CI, deployment and CodeQL
pipelines/               Azure Pipelines alternative
scripts/                 Deployment, state bootstrap, PDF build
```

Detailed walkthroughs: docs/00-start-here.md through docs/06-foundry-demo.md.

Prepare a safe local workspace

- Python 3.10+, Node.js 22+, Git and curl.
- Docker with a running Linux-container daemon for the container labs.
- Terraform 1.5 to below 2.0; jq for deployment regression tests.
- Start commands at the repository root unless instructed otherwise.
- Use synthetic customer data; keep tokens, state and ticket access keys private.
- Python environment: support-agent/backend/.venv (not support-agent/.venv).

Do not erase someone else's changes or kill an unrelated process to free a port.

Lab 1: build the UI and run Flask

```
(cd support-agent/frontend && npm ci && npm run build)
python3 -m venv support-agent/backend/.venv
support-agent/backend/.venv/bin/pip install \
  -r support-agent/backend/requirements.txt
cd support-agent/backend
export AI_MODE=off STORAGE_ACCOUNT_URL=''
export AZURE_STORAGE_CONNECTION_STRING=''
export SPECIALIST_API_TOKEN="$(.venv/bin/python -c \
  'import secrets; print(secrets.token_urlsafe(32))')"
.venv/bin/python app.py
```

Open http://127.0.0.1:8000/ ; stop the server with Ctrl+C.

Lab 2: React hot reload

Keep the backend running on port 8000. Open a second terminal at the repo root:

```
cd support-agent/frontend
npm ci
npm test
npm run dev -- --host 127.0.0.1
```

- Open the Vite URL shown in the terminal, normally port 5173.

- Vite proxies /api and /health to the Flask backend on port 8000.

- npm run build produces dist/ for Flask and the production container.

Customer route: #/ | Specialist route: #/specialist.

Follow a ticket through the backend

- app.py: app factory, routes, bearer authentication and JSON errors.
- tickets.py: input validation, ticket access, approval and reassignment.
- support_agent.py: specialist routing and AI/template drafts.
- knowledge.py: parse Markdown sections, cache and rank matches.
- storage.py: BlobStore in Azure; LocalStore for offline development.
- ai_client.py: off, model or agent; manage_agent.py: Terraform agent lifecycle.

Trace the code in docs/01-python-explained.md before changing behavior.

API contract: customer and specialist

```
GET  /health                          liveness
GET  /api/config                      AI mode and four teams
POST /api/tickets                     returns ticket + access_key
GET  /api/tickets/{id}                 header: X-Ticket-Key
GET  /api/specialist/tickets           header: Authorization: Bearer ...
POST /api/specialist/tickets/{id}/approve
     {"response": "The reviewed answer"}
POST /api/specialist/tickets/{id}/reassign
     {"category": "technical"}
GET  /api/specialist/knowledge
```

Drafts are private. Customer views exclude them until the response is approved.

Four teams, one review queue

- billing: Billing & Payments | Aye Chan
- technical: Technical Support | Min Thu
- account: Account & Security | Su Mon
- general: General Enquiries | Kyaw Zin
- Priority is high or normal. Specialists can filter and reassign tickets.
- Shared token enables this teaching console; it is not per-person authentication.

Confirm routing and knowledge sources before approving an answer.

Knowledge source: Markdown to Blob

- Four files in support-agent/knowledge cover billing, technical, account and general queries.
- Front matter supplies title and category; level-two headings define retrievable sections.
- Terraform uploads the Markdown files into the private knowledge container.
- The app ranks knowledge sections and passes excerpts to the AI or template draft.
- The console displays the draft's source references for the reviewer.
- Locally the folder store is seeded automatically from the same source files.

Edit knowledge in source control; review the change and refresh the stored documents.

Ticket persistence and concurrency

- Azure stores one JSON blob per ticket in the tickets container.
- The customer access key is returned at creation; only its hash is stored.
- The browser keeps ticket IDs and keys in localStorage for tracking.
- Updates use Blob ETags: a concurrent write is rejected with HTTP 409.
- Versioning and seven-day soft delete help recovery; they are not a complete backup policy.
- LocalStore is a single-process teaching fallback; use Blob Storage for Azure.

Storage for app data is separate from the Terraform state container.

Lab 3: verify the real workflow

- Submit: "I was charged twice for my invoice."
- Confirm billing / Aye Chan and a pending customer view.
- Open #/specialist and privately enter the server's specialist token.
- Read the knowledge sources, edit the draft and select Approve & send.
- Confirm the exact approved text appears in the customer portal.
- Create another ticket and reassign it to check the second path.

Never paste the token or a customer's access key into logs, chat or screenshots.

Tests: fast evidence before packaging

From the repository root:

```
(cd support-agent/backend && .venv/bin/python -m unittest -v)
(cd support-agent/frontend && npm test && npm run build)
bash -n scripts/deploy-azure.sh tests/test-deploy.sh
bash tests/test-deploy.sh
```

- Backend tests cover API, ticket lifecycle, knowledge, storage and mocked AI.
- Frontend tests exercise customer and specialist flows.
- Deployment regressions mock Azure and HTTP; they do not prove live permissions.

Distinguish unit evidence, container evidence and a live Azure rehearsal.

Docker: build two images from one source

- Stage 1: node:22-alpine installs locked npm dependencies and builds React.
- Backend target: python:3.12-slim runs Flask/Gunicorn as non-root UID 10001.
- Frontend target: Node serves React and proxies API/health to BACKEND_ORIGIN.
- Health check calls /health. No secrets or development environments enter the image.
- Local data lives under /data; Azure selects Blob Storage using its endpoint.

```
docker build --target backend -t mmaug-support-agent-backend:1.0 support-agent
docker build --target frontend -t mmaug-support-agent-frontend:1.0 support-agent
```

The default combined target is retained for the single-container foundations lab.

Lab 4: Compose with persistent local data

Reuse the specialist token from your server terminal, or generate a fresh one:

```
export SPECIALIST_API_TOKEN="$(python3 -c \
  'import secrets; print(secrets.token_urlsafe(32))')"
docker compose -f support-agent/compose.split.yaml up --build -d
curl -fsS http://127.0.0.1:8090/health
docker compose -f support-agent/compose.split.yaml logs --tail=50
docker compose -f support-agent/compose.split.yaml down
```

- Open the portal and console on port 8090.

- The named volume survives down. Adding -v deletes local ticket data.

Compose uses offline drafting; do not hard-code credentials into YAML.

Lab 5: declare the local container

Build the image from the previous lab first. With a specialist token exported:

```
cd support-agent/terraform
export TF_VAR_specialist_token="$SPECIALIST_API_TOKEN"
terraform init
terraform plan -out=local.tfplan
terraform apply local.tfplan
# Visit http://127.0.0.1:8080 and verify the workflow.
terraform destroy
```

- Terraform declares the Docker image, container, loopback port and named volume.

- The token is sensitive but still stored in state. Keep state and plans private.

Review the destroy plan: it also removes this lab's data volume.

Azure Terraform: ownership and naming

- provider.tf declares providers; dev.tfvars holds placeholder development values.
- main.tf reads existing RG-MMAUG-Bootcamp; it must never own/delete the group.
- Terraform owns two Linux web apps, their plan, ACR, Foundry and Search.
- Bootstrap owns the storage account and state / knowledge / tickets containers.
- foundry.tf optionally provisions the AI account, project, model and prompt agent.
- Infrastructure workflow bootstraps before init; app delivery owns image digests and token.

Preserve state ownership. Never use a fresh state file to manage existing resources.

Blob Storage and runtime identity

- StorageV2 account: HTTPS, minimum TLS 1.2, private containers, no shared keys.
- App identity: Storage Blob Data Contributor on tickets; Reader on knowledge.
- Pipeline: approved data access for bootstrap/state and knowledge ingestion.
- App identity also has AcrPull and the configured Foundry/model permissions.
- DefaultAzureCredential uses the web app's managed identity in Azure.
- Search identity reads knowledge; agent identity queries the index. No app state access.

Management-plane Contributor alone does not necessarily grant Blob data access.

Plan first: existing-state migration

Do not run apply as a prerequisite for reading the code.

```
cd infra/azure
terraform init -backend=false
terraform fmt -check -recursive
terraform validate
# Use infra.yml for authenticated remote-state plan/apply.
```

- Review subscription and prefix locally; never commit actual environment values.
- Migrate earlier state and review RG/app/storage ownership before the first pipeline apply.
- Agent creation needs the backend Python dependencies and authorized Azure access.

Back up existing state and review every proposed deletion before authorizing apply.

Terraform state and cleanup

- Remote state is independent of application tickets and knowledge.
- infra.yml runs scripts/bootstrap-state.sh before Terraform initialization.
- Bootstrap storage remains outside workload Terraform destroy ownership.
- Secure state storage with Entra ID; state and plans can contain sensitive values.
- Preserve state and configuration until the owned resources are destroyed.
- App Service, Search, ACR, Storage and AI incur charges even without workshop traffic.

```
# Explicitly authorized cleanup, from infra/azure:
terraform plan -destroy -var-file=dev.tfvars
terraform destroy -var-file=dev.tfvars
```

Never delete the shared resource group. Retain state until owned workload resources are gone.

AI drafting: three modes

- AI_MODE=off: retrieve knowledge and produce a template draft; offline and free.
- AI_MODE=model: call the Azure OpenAI deployment directly.
- AI_MODE=agent: invoke the Foundry prompt agent mmaug-support-agent.
- Cloud grounding indexes Blob knowledge in Azure AI Search / Foundry IQ.
- The expected JSON has category, priority and reply; invalid output or AI errors use the template fallback.
- The specialist reviews every mode; the model never approves or sends a response.

Paid AI is optional. Check region, model availability, quota and cost first.

Foundry: configuration and evidence

- enable_foundry controls paid Terraform resources; ai_mode selects agent or model.
- FOUNDRY_PROJECT_ENDPOINT and FOUNDRY_AGENT_NAME configure agent calls.
- AZURE_OPENAI_ENDPOINT, DEPLOYMENT and API_VERSION configure direct model calls.
- manage_agent.py is the Terraform data-plane bridge for agent creation/deletion.
- The existing project's resource name is retained for an in-place update.
- Offline tests mock SDK calls; a live authorized demo must verify identity, model access and fallback.

Read docs/06-foundry-demo.md; do not assume a green mock test proves a live AI call.

GitHub infrastructure: bootstrap before init

- Use existing mmaug_bootcamp_spn_2026 with OIDC and environment dev.
- An administrator prepares scoped RBAC administration and Blob data access.
- Bootstrap creates/reuses the account with tfstate, knowledge and tickets containers.
- Read existing RG-MMAUG-Bootcamp; never create or destroy the shared group.
- Review the saved Terraform plan and apply only through the authorized boundary.
- Verify IQ indexing and the managed MCP connection before registering the agent.
- Migrate any previous state before the first remote-backend run.

OIDC is authentication, not approval. Set required reviewers and branch restrictions.

GitHub Actions: CI to approved delivery

- ci.yml: PR/main checks for backend, frontend, container, Terraform and deployment scripts.
- A successful push to main calls deploy.yml automatically; PRs do not deploy.
- deploy.yml tests and releases via dev; configure external approval rules before use.
- OIDC authenticates to Azure without a client secret.
- Build backend/frontend, push unique tags to ACR and deploy both resolved digests.
- CodeQL runs separately; it is not automatically an alerts-based release gate.

Required reviewers and branch restrictions must be configured outside the YAML.

GitHub: variables and release controls

- Environment variables: AZURE_CLIENT_ID, AZURE_TENANT_ID, AZURE_SUBSCRIPTION_ID.
- Also set AZURE_RESOURCE_GROUP, ACR_NAME, BACKEND_WEB_APP_NAME and FRONTEND_WEB_APP_NAME.
- Environment secret: SPECIALIST_API_TOKEN, random and at least 32 characters.
- Existing mmaug_bootcamp_spn_2026 trust targets this repository's dev environment.
- Use scoped ACR push and App Service deployment permissions.
- Manual deploy runs require the same authorization and reviewed CI evidence as automatic releases.

Never trigger a workflow or push main merely to test documentation.

Azure Pipelines: an alternative, not a second owner

- Definition: pipelines/azure-pipelines.yml.
- Variable group: mmaug_bootcamp_var at the top-level variables block.
- Workload-identity-federated service connection: azure-demo-wif.
- Stages: Test -> Build image/digest -> approved Deploy to App Service.
- Secret SPECIALIST_API_TOKEN is explicitly mapped to the deployment environment.
- deploy defaults true; select false for a test-only run.
- Its Test stage does not mirror every GitHub CI check; validate Terraform/container separately.

Select one orchestrator for the shared target; their locks do not coordinate.

Deployment verification and recovery

- scripts/deploy-azure.sh accepts immutable backend and frontend image digests.
- It sets the backend token, frontend BACKEND_ORIGIN and each app's image.
- Verify /health, /api/config with four teams and authenticated /api/specialist/knowledge.
- The knowledge check proves Blob read access, not ticket writes or approval correctness.
- Check both apps and React shell; a failure can leave only one app updated.
- Recovery requires a reviewed known-good digest and explicit release authorization.

Complete a synthetic ticket-and-approval rehearsal after the automated checks.

DevSecOps: what CodeQL does and does not do

- codeql.yml scans Python, JavaScript/TypeScript and GitHub Actions.
- codeql-config.yml enables security-extended queries and excludes generated/vendor files.
- Inspect alerts, distinguish true findings and record a justified disposition.
- Security headers, hashed access keys, bounded requests and non-root runtime provide other controls.
- OIDC/WIF and least-privilege RBAC reduce credential handling.
- A successful scan is not a dependency audit, penetration test or automatic approval to release.

Configure branch protections and review policies deliberately; do not infer them from YAML.

Production gaps: state them honestly

- One shared specialist token: replace with per-user identity and authorization.
- No public ticket rate limit: paid AI can be abused; add quotas before public use.
- Portal-only responses: email delivery would need an additional integration.
- Storage is authenticated but public-network accessible; private networking is out of scope.
- LocalStore is for a single process, not a shared production database.
- Extend audit, retention, backup and monitoring policies for real customer data.

This is a small teaching app, not a production-ready support platform.

Workshop exit checklist

- A customer ticket is triaged and assigned to the correct specialist.
- Knowledge sources are visible; no unapproved draft leaks to the customer.
- Approval and reassignment work; customer sees the reviewed response.
- Tests and the image build pass; plans and roles have been reviewed.
- Authorized releases use one orchestrator, environment controls and an immutable digest.
- Cleanup is scoped and approved; no secrets, state or customer data enter Git.

Reusable runbook: skills.md | Architecture: README.md | Detailed labs: docs/00-06.